[CE310] Evolutionary Computation and Genetic Programming

Riccardo Poli

Bloat in GP

University of Essex
School of Computer Science and Electronic Engineering

What did we learn last time? (1)

- We learned that different program execution strategies exist focusing on interpretation.
- We have seen that interpreting a **program tree** means **recursively traversing** the tree and **executing nodes** when their arguments are known.
- We have learned that there are problems with **non-functional** instructions and that we need **macros**, which have unevaluated arguments as their input.

What did we learn today? (2)

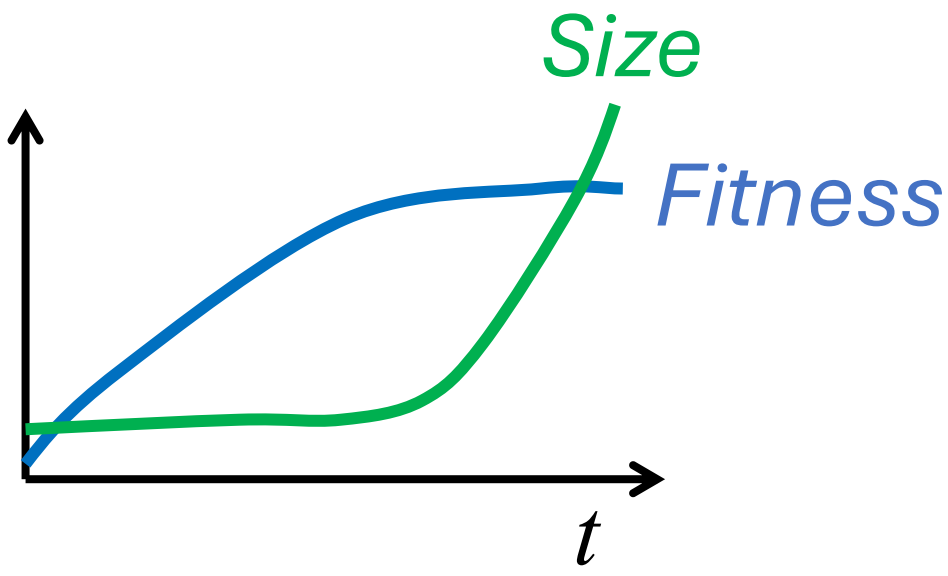
- XO with tree-like programs requires **swapping** of sub-trees, mutation requires replacing a sub-tree with a new random one.
- Programs can also be composed of **multiple trees**, each representing a different component of the program.
- This can be used to define subroutines (ADFs) that may **reduce** the **complexity** and increase **generality** and **understandability** of solutions.

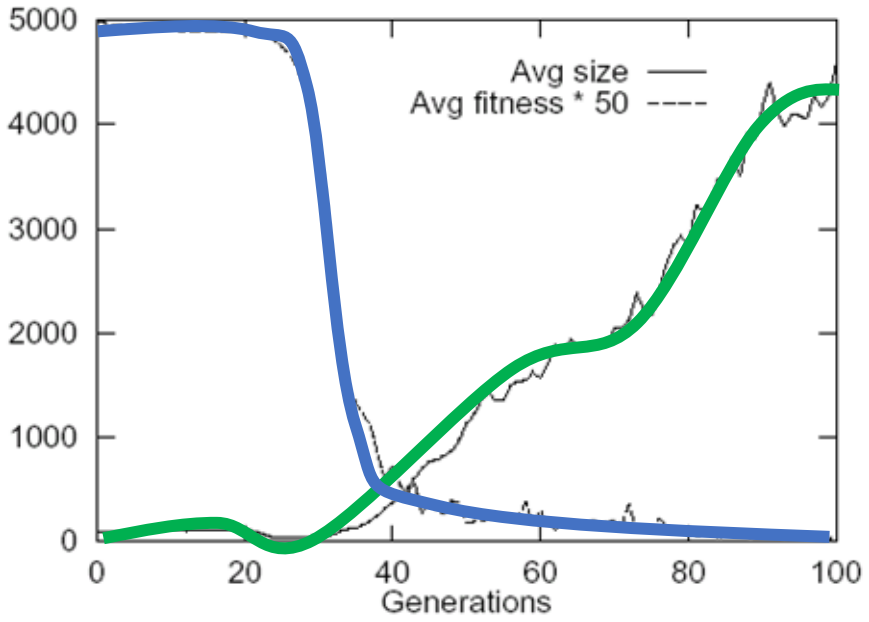
This week’s learning objectives

- Describe **bloat**
- Describe different **theories** that explain “**why bloat happens?**” and discuss them.
- Explain approaches to **control** bloat

What's bloat?

Bloat = **Growth** in **program size** (variable length representation) without **significant** return in terms of **fitness**.

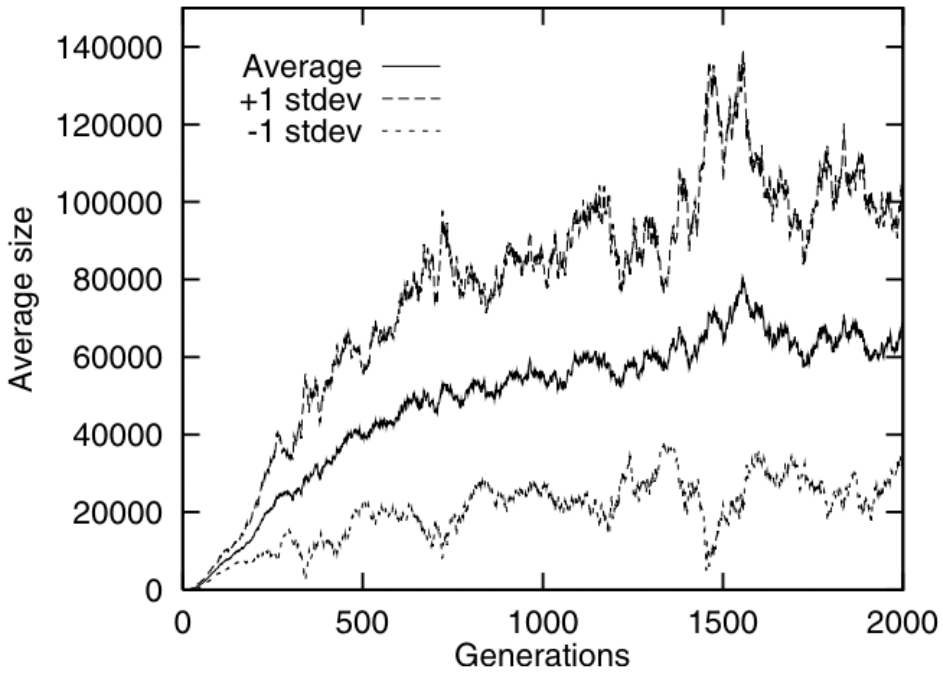




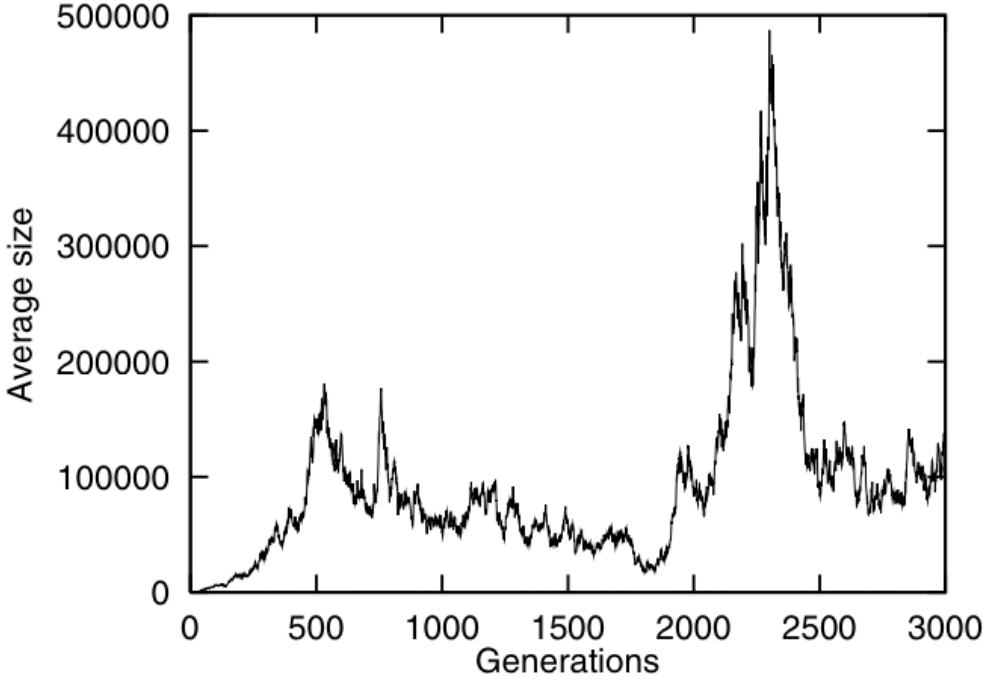
McPhee & Miller (1995), McPhee & Poli (2001)

Does it continue forever?

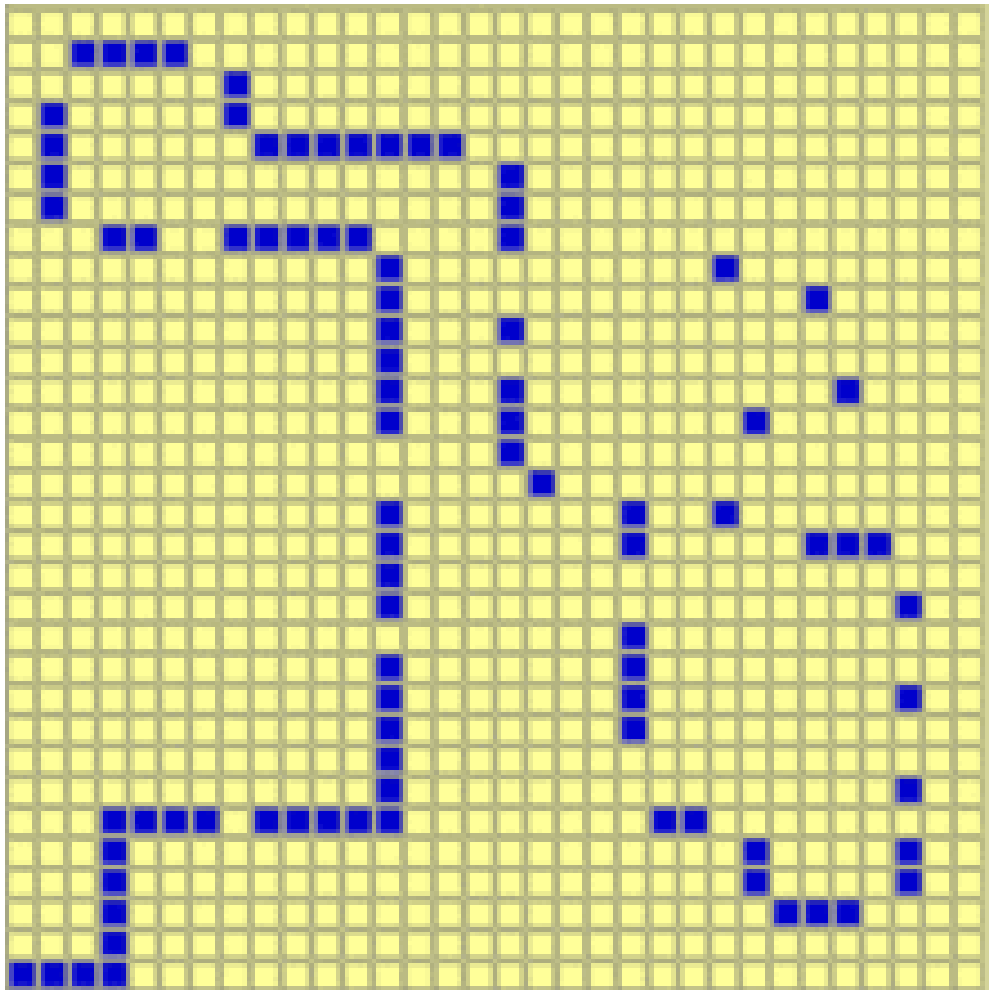
Average behaviour



Single run



Artificial ant problem (Santa Fe trail)



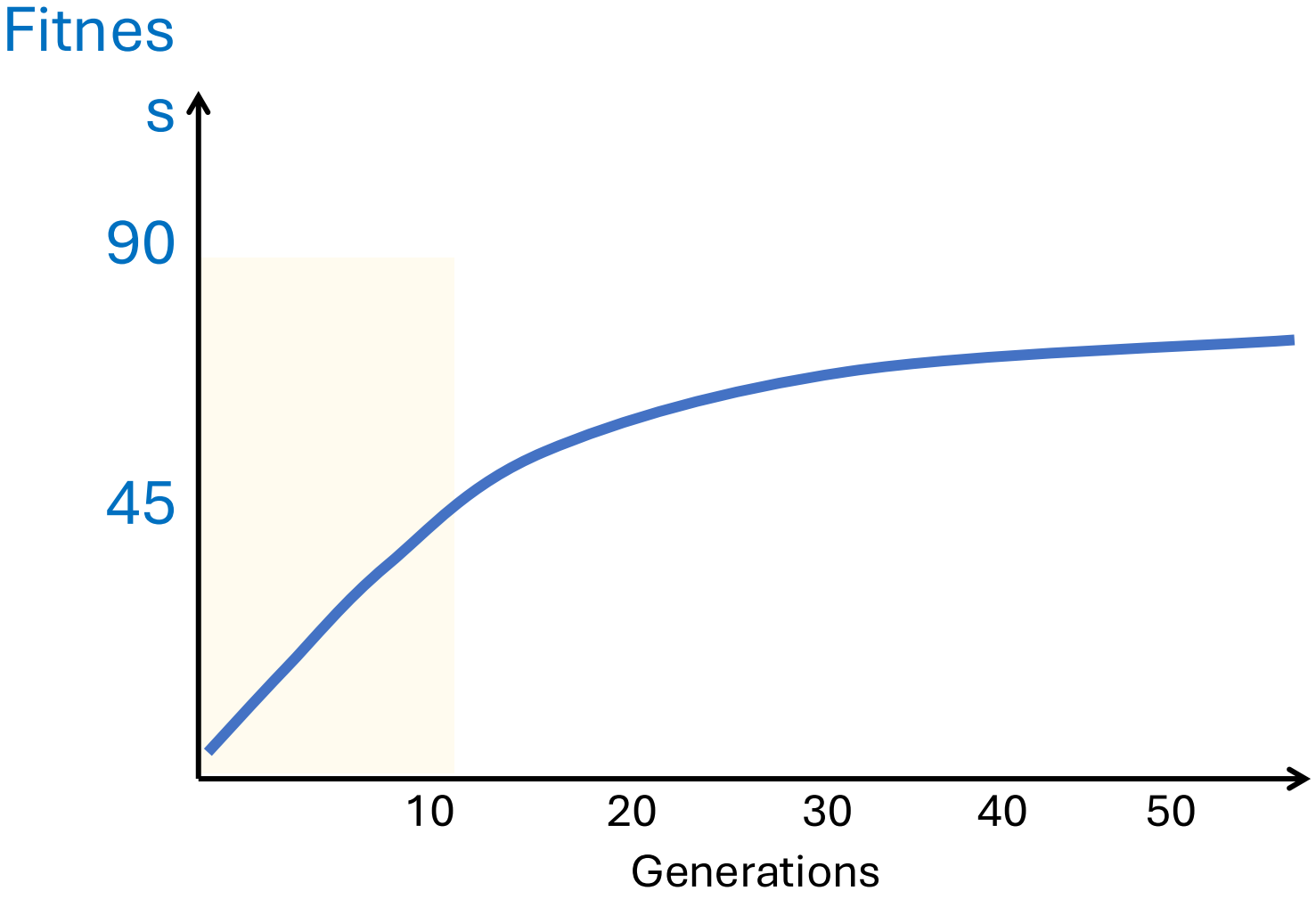
Artificial ant problem (Santa Fe trail)

Objective:	Find an ant that follows the "Santa Fe trail"
Terminal set:	Left, Right, Move
Functions set:	IfFoodAhead, Prog2, Prog3
Fitness cases:	The Santa Fe trail
Fitness:	Food eaten
Selection:	Tournament group size of 7, non-elitist, generational
Wrapper:	Program repeatedly executed for 600 time steps.
Population Size:	500
Max program:	500
Initial population:	Created using "ramped half-and-half" with a maximum depth of 6
Parameters:	90% one child crossover, no mutation. 90% of crossover points selected at functions, remaining 10% selected uniformly between all nodes.
Termination:	Maximum number of generations G = 50

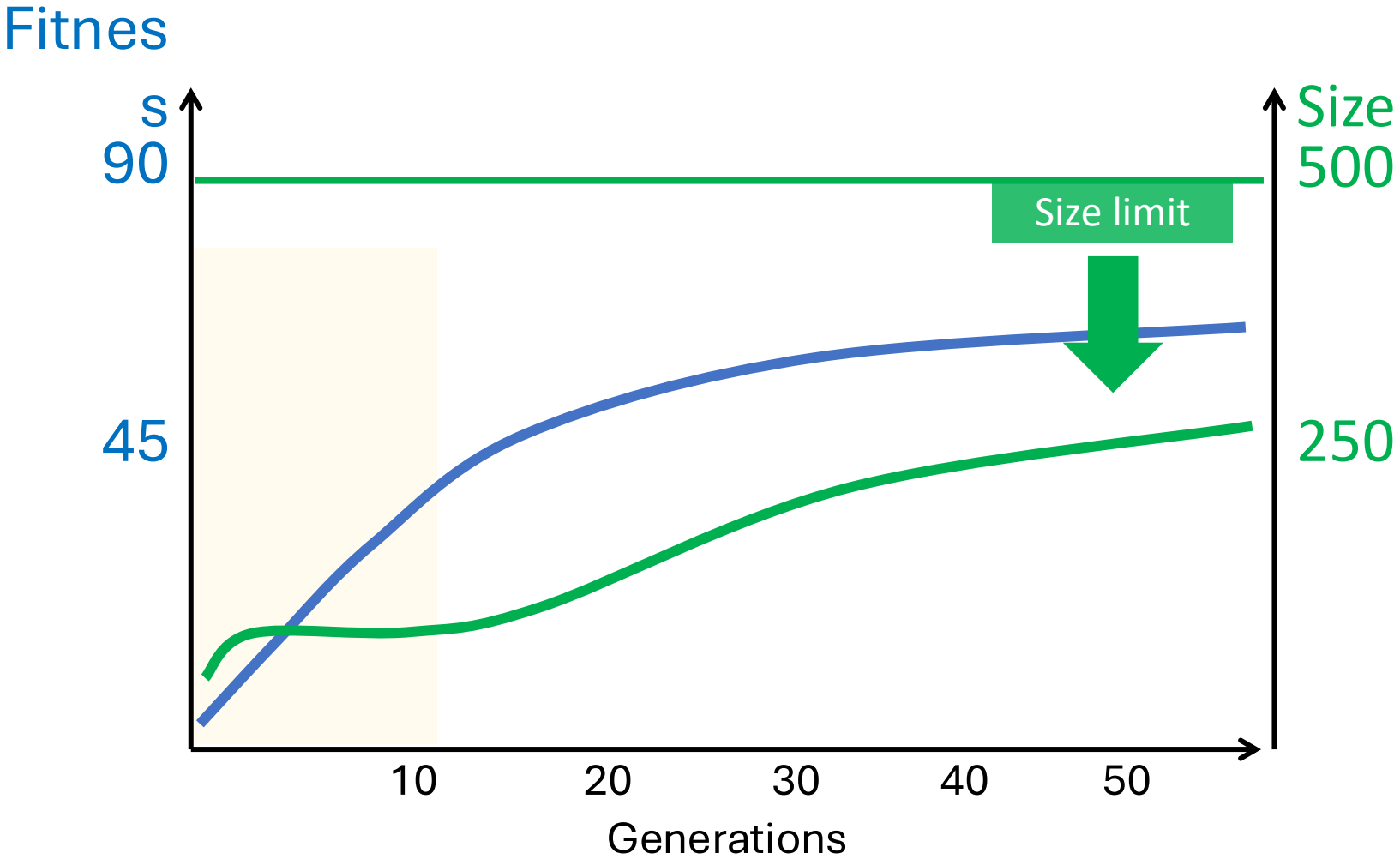
Langdon & Poli (1998)

Santa Fe trail problem: https://www.youtube.com/watch?v=BKF7pGw8qbY

Ant Problem: Mean Fitness vs Generations



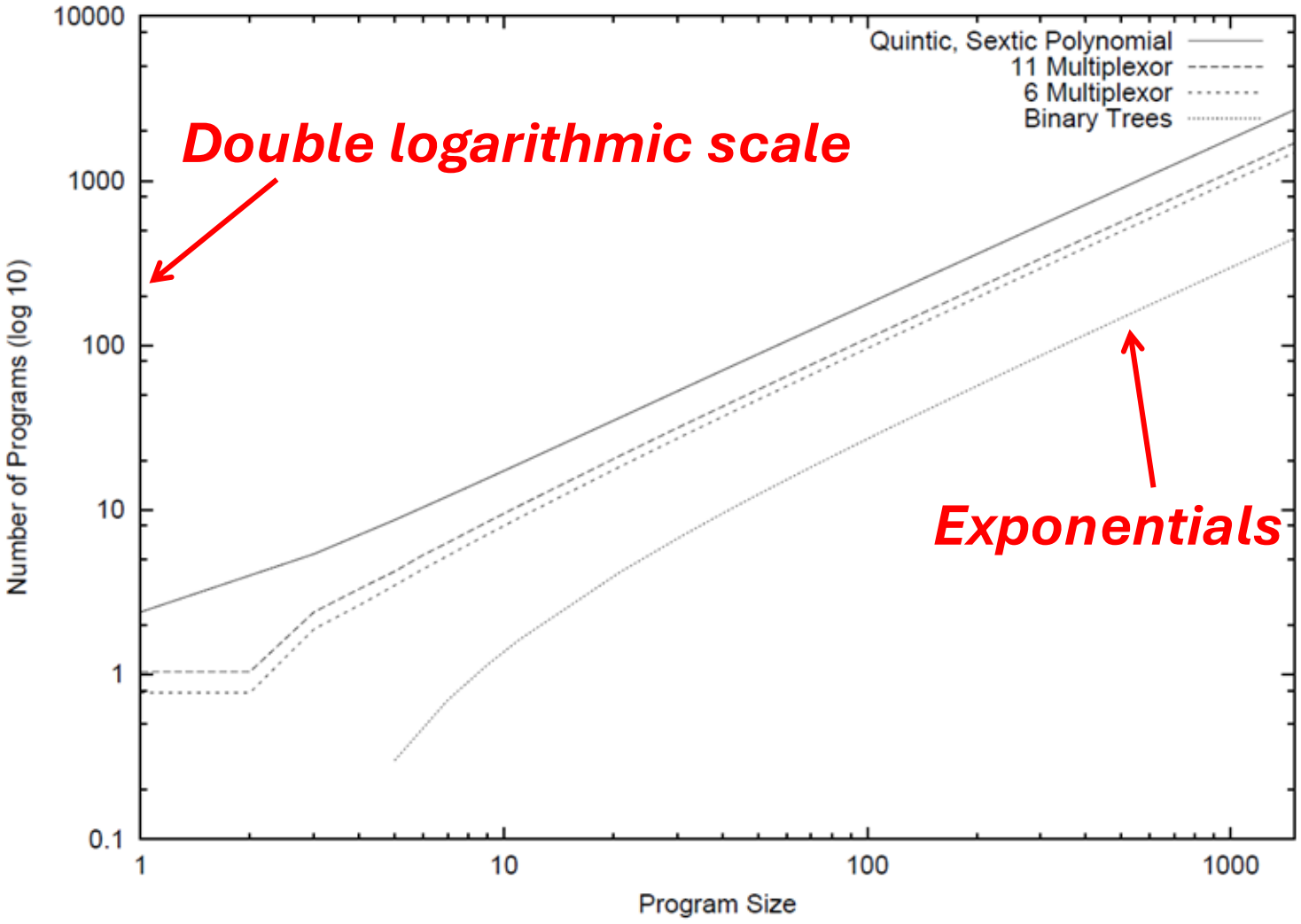
Ant Problem: Mean Program Size vs Gen.



Bloat - Why does it happen?

- Bloat has been thoroughly investigated for decades
- (Too) many theories propose to explain it.
- So, for some people bloat is still a mystery.

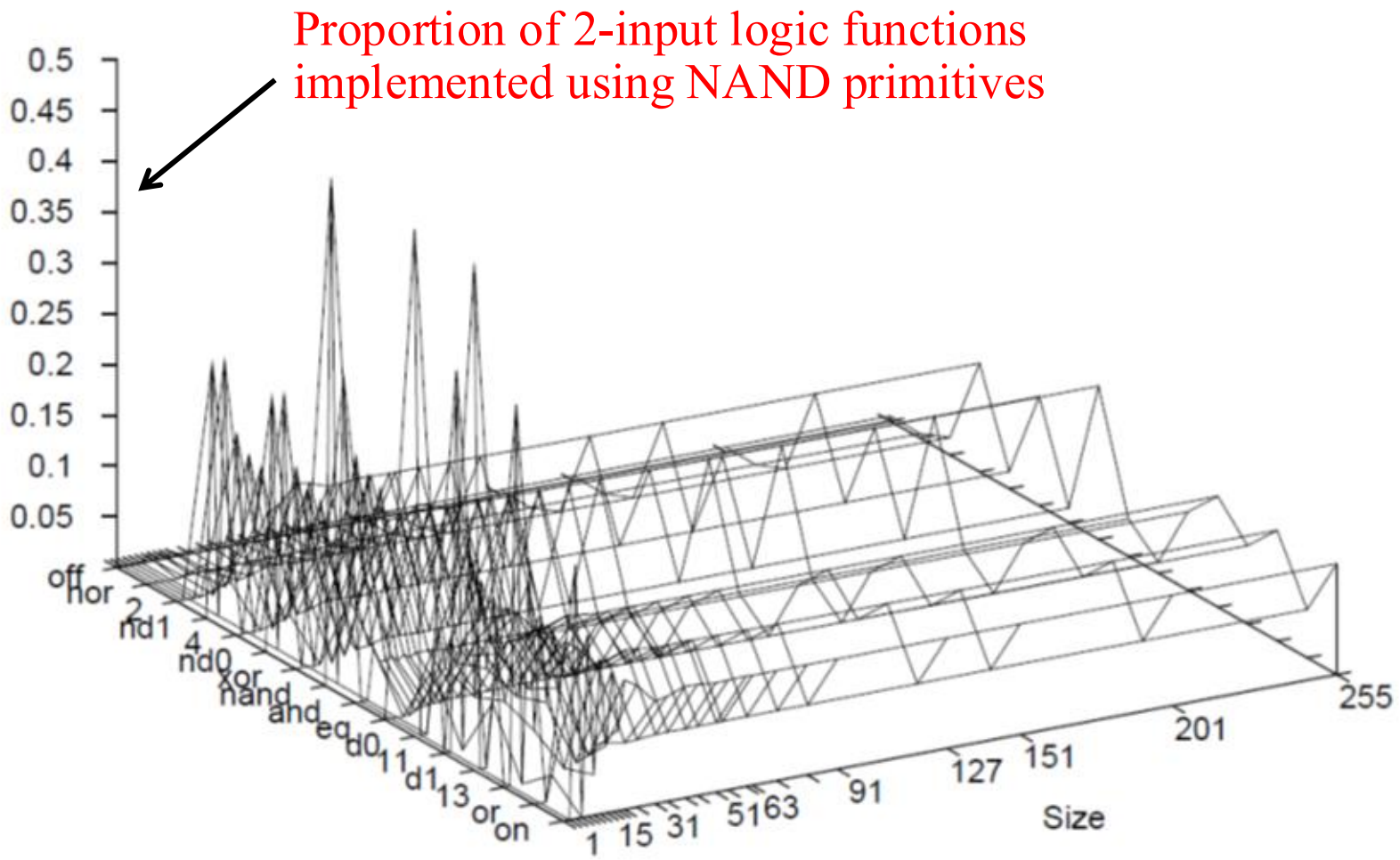
GP Search Space



Limiting distribution

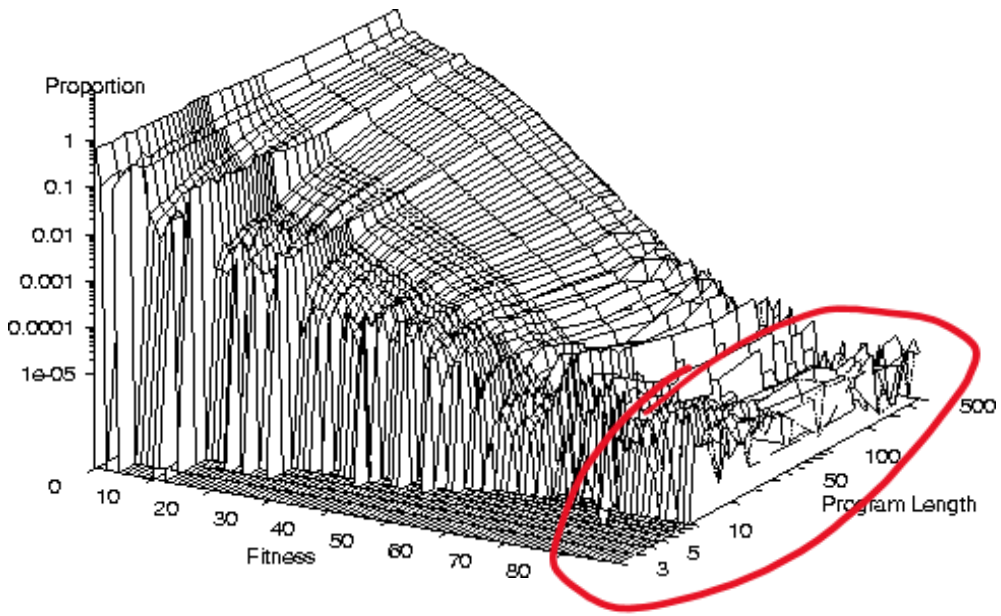
- Empirically and theoretically, it has been shown that **as program length grows the distribution of functionality reaches a limit**

{d0,d1,NAND} search space

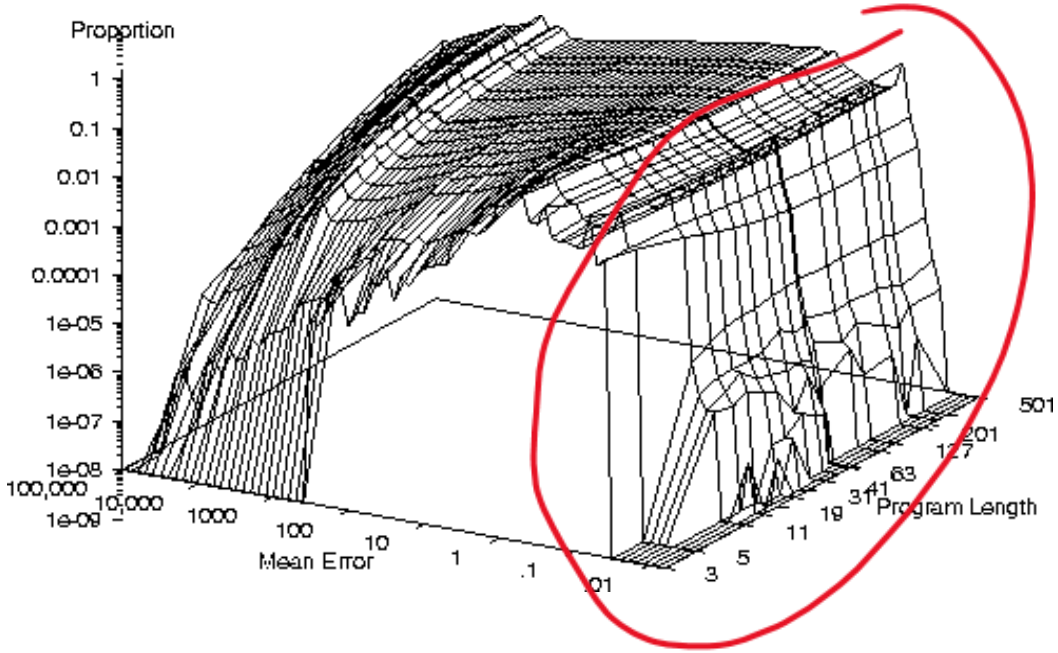


Search spaces

- Proportion of Ant programs with each score



- Error Distribution Sextic Polynomial Problem (x^6+...)

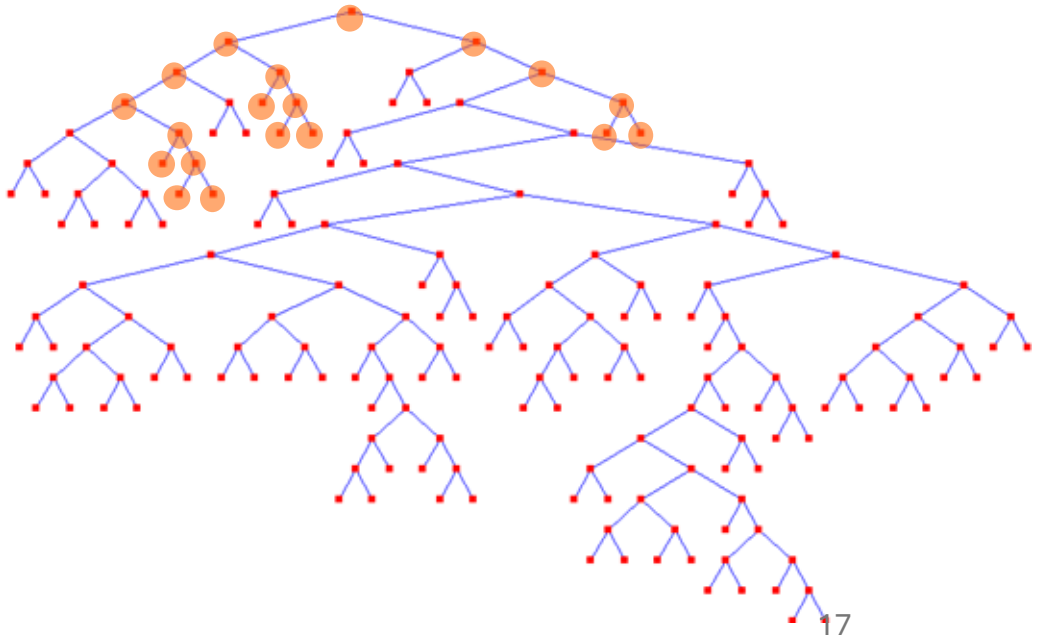


Nature of program search spaces theory

- Above a certain size, the **distribution of fitness does not vary with size**.
- Since there are **more long programs**, the number of long programs of a given fitness is greater than the number of short programs of the same fitness.
- Thus, over **time GP samples longer and longer programs** simply because there are more of them.

Active vs Inactive Code

- **Most code** in bloated programs is **inactive**, e.g.
  - *a* * 0 = 0 however big the expression for *a* is
  - IF(0, *b*, *c*) = *c* whatever *b* is
- When a **fit program** is **changed**, it **produces unfit offspring most** of the **time**.
- Active code is like "**icing on the cake**"



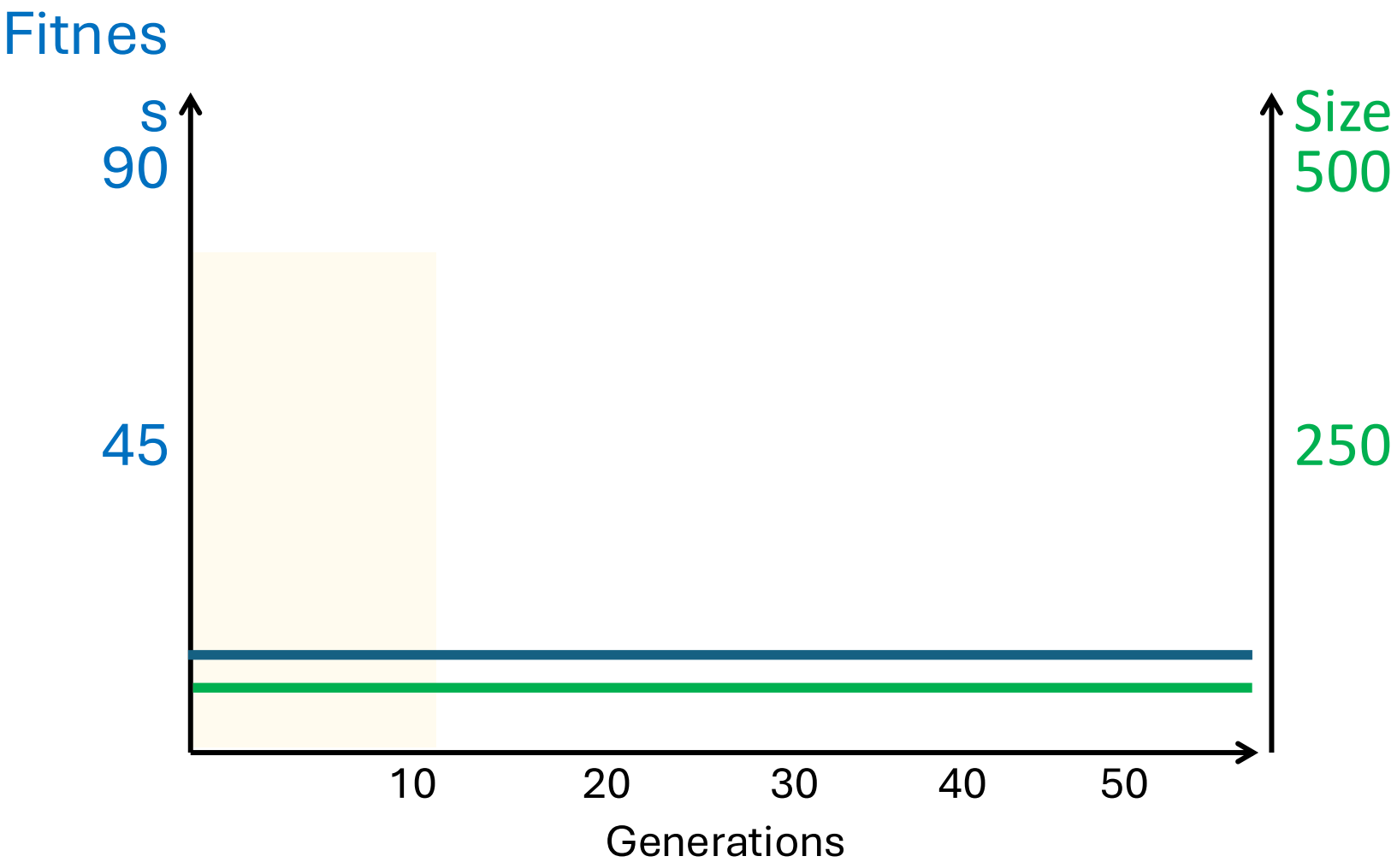
Replication accuracy theory

- The success of a GP individual depends on its ability to have **offspring that are functionally similar** to the parent.
- So, **GP evolves towards (bloated) representations that increase replication accuracy**.

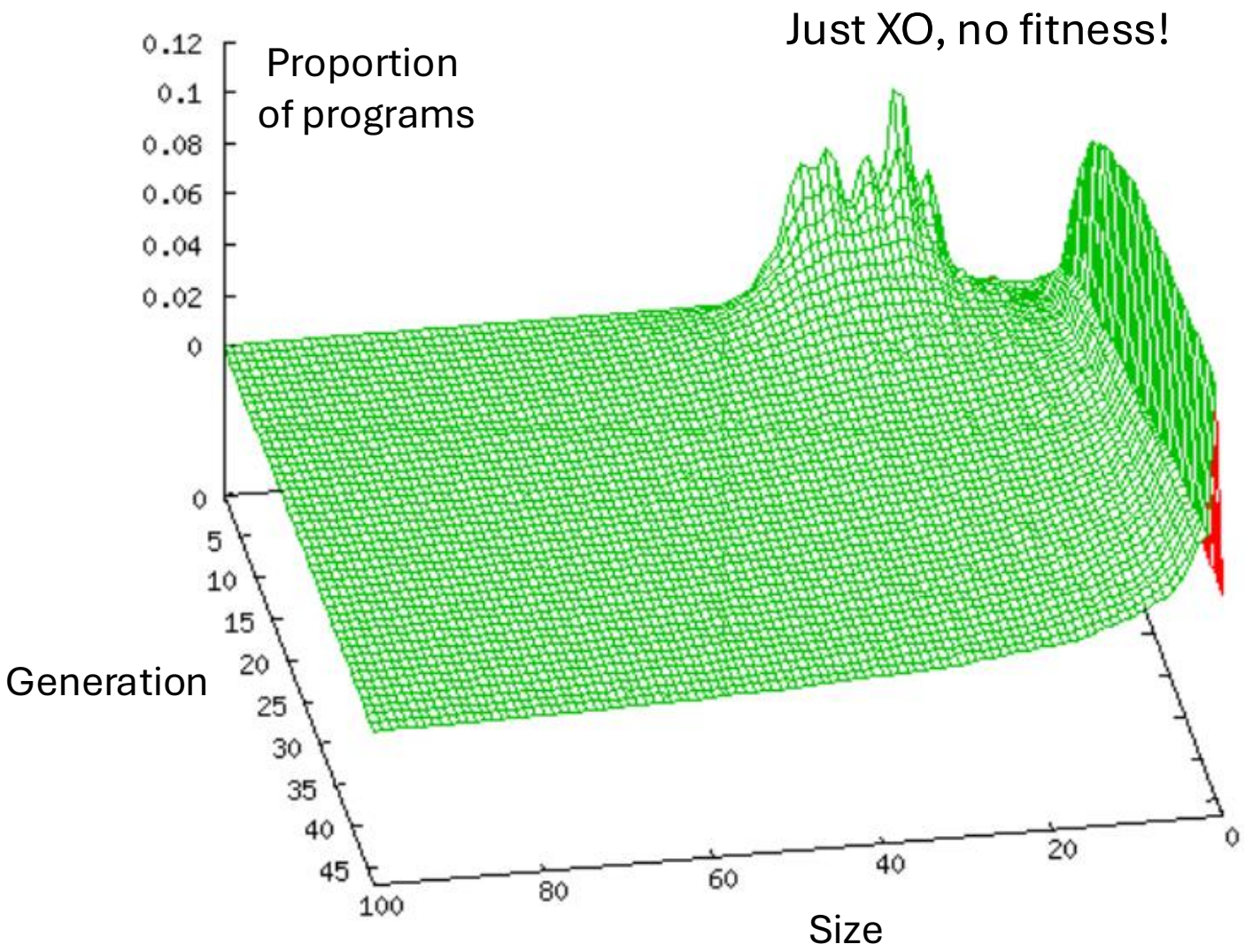
Removal bias theory

- **Inactive code** in a GP tree is low in the tree, forming **smaller-than-average-size subtrees**.
- Crossover events **excising inactive subtrees** produce offspring with the **same fitness** as their parents.
- On average the **inserted subtree is bigger** than the excised one, so such offspring are bigger than average.

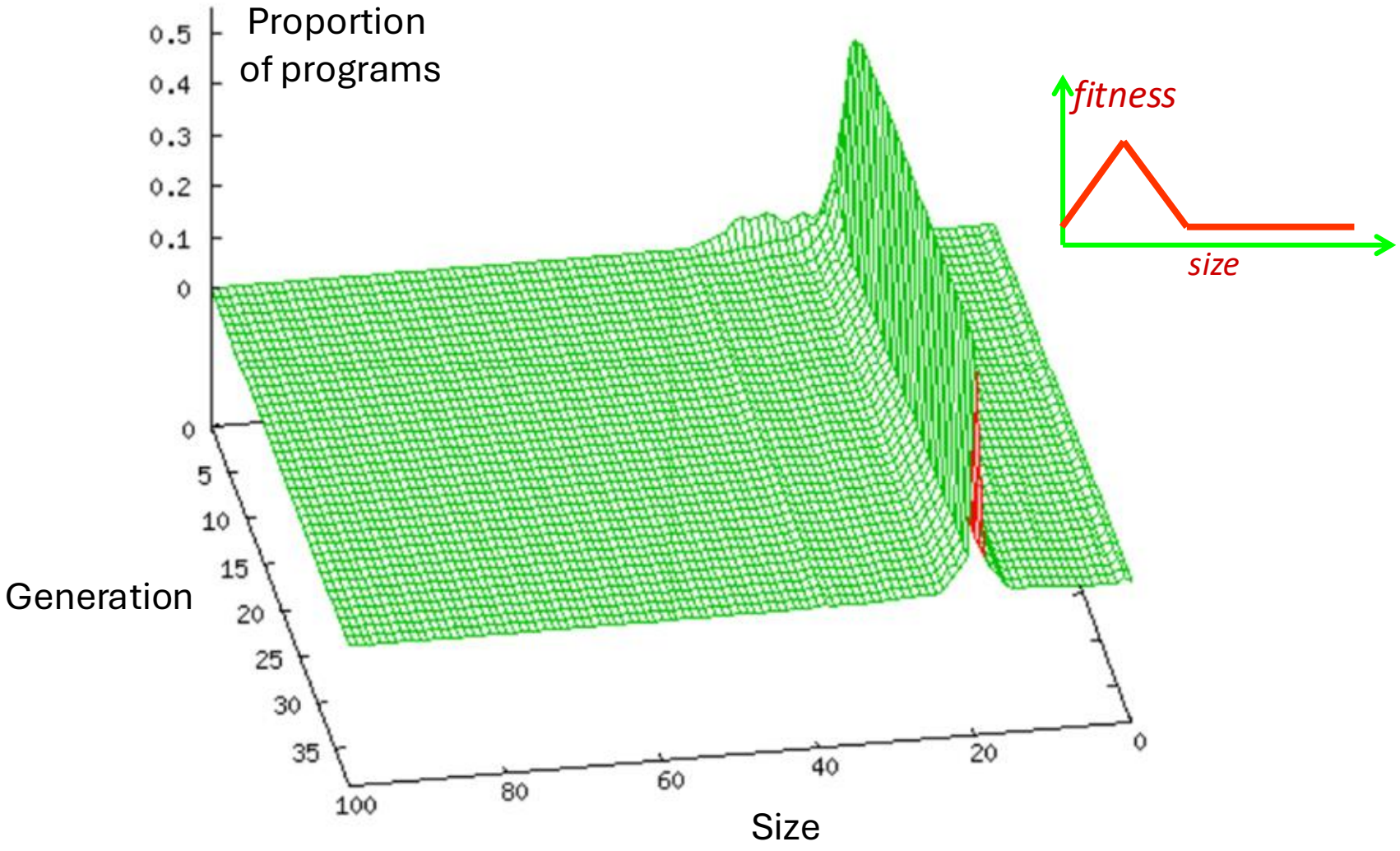
More evidence: No Fitness = No Bloat



Crossover Bias



Can fitness override the XO bias?



Crossover Bias Theory of Bloat

- **Crossover does not change the mean program size**, on average, but…
- It creates a population with a **large proportion of small programs**.
- In most problems, **very small programs are unfit**, so they are ignored by selection. Thus…
- Only **larger programs will be picked as parents**, hence …
- **Mean program size increases**.

Size Evolution (1)

- The mean size of the programs in the current generation is

$$\mu_g = \frac{1}{N}\sum_{i=1}^{N} s_i$$

  with N number of programs

- However, if we group programs by size:

$$\boldsymbol{\mu_g = 1 \times \Phi(1) + 2 \times \Phi(2) + \cdots = \sum s \times \Phi(s)}$$

  with $\boldsymbol{\Phi(s)}$ = proportion of programs of size $\boldsymbol{s}$ in the population

Size Evolution (2)

- Instead of creating a new generation ***g+1*** by selection and crossover, we split this into two steps:
  1. we create a new generation just by selection and duplication, and
  2. create a further generation by applying crossover (randomly selecting parents from the first).
- After (1), the mean program size will be

$$\boldsymbol{\mu_{sel}} = \mathbf{1} \times \mathbf{p}(\mathbf{1}) + \mathbf{2} \times \mathbf{p}(\mathbf{2}) + \cdots$$

  where **p(s)** = probability of selecting a program of size **s** from the population
- On average, **crossover on its own takes away as much as it puts in**.
- So going from (1) to (2) we should not see any changes in mean program size. That is

$$\boldsymbol{\mu_{\mathbf{g+1}}} = \boldsymbol{\mu_{sel}} = \mathbf{1} \times \mathbf{p}(\mathbf{1}) + \mathbf{2} \times \mathbf{p}(\mathbf{2}) + \cdots$$

Conditions for Growth

- Growth/bloat can happen only if

$$\mathbf{E}\left[\boldsymbol{\mu}_{\mathbf{g+1}} - \boldsymbol{\mu}_{g}\right] = 1 \cdot [p(1) - \Phi(1)] + 2 \cdot [p(2) - \Phi(2)] + \cdots + \mathrm{N} \cdot [p(\mathrm{N}) - \Phi(\mathrm{N})] > \mathbf{0}$$

- So, there must be:
  - **some short programs for which** $\boldsymbol{p(s) < \Phi(s)}$ or
  - **some longer ones for which** $\boldsymbol{p(s) > \Phi(s)}$ (or both, at least on average).
- For years, the focus was on why are longer programs were preferred?

  ($\boldsymbol{p(s) > \Phi(s)}$ for big $\boldsymbol{s}$)
- The **crossover bias theory** (and my experiments with Nic McPhee) **showed that the culprit was the short/bad programs!**

  ($\boldsymbol{p(s) < \Phi(s)}$ for small $\boldsymbol{s}$)

Main techniques for limiting code bloat: Size limits

- **Fixed size or depth limit**: Programs exceeding the limit are discarded and a parent is kept instead.
- This is **very dangerous** as it gives a fitness advantage to programs that tend to violate the constraint.
- Trying crossover again or giving 0 fitness to programs above the threshold is a better way to do it

Main techniques for limiting code bloat: Modifying operators

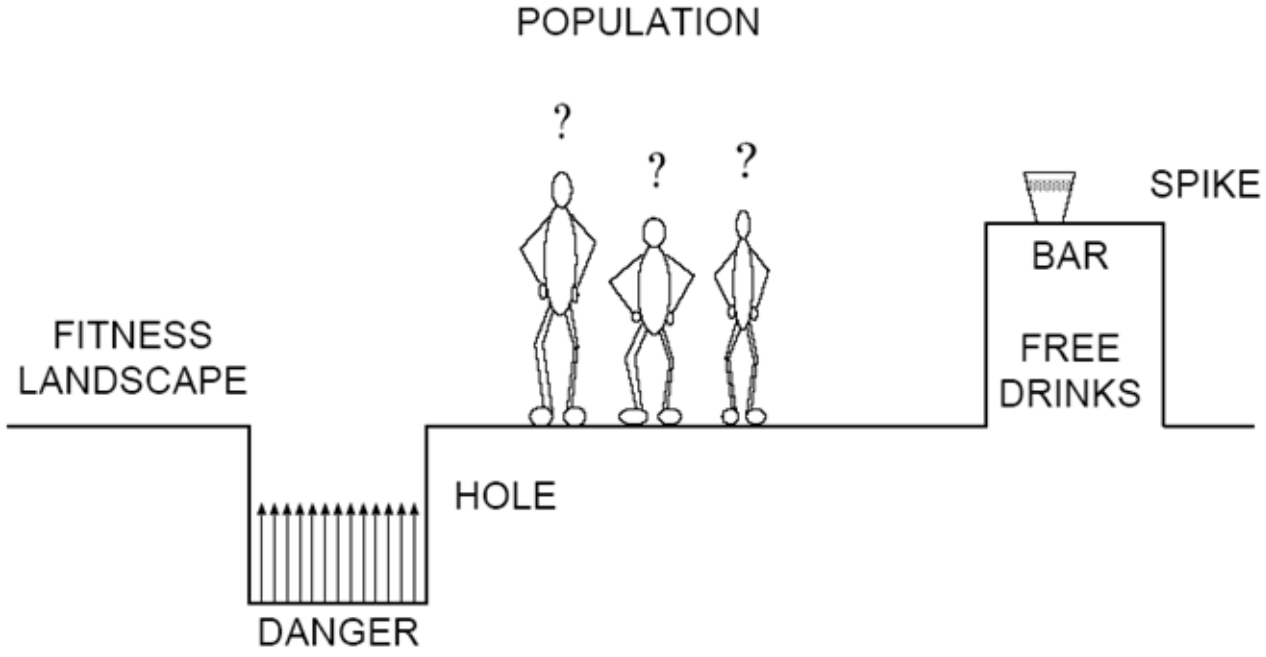
- **Modification of operators**: variation of the selection probability of crossover points by using explicitly defined introns, rejection of destructive crossover events, size-fair operators, multi-objective optimization (MOO) techniques, etc.
- For example, **point mutation** (applied with a fixed probability per node) has an anti bloat effect.

Main techniques for limiting code bloat: Acting on Selection

- Size evolution equation shows that to prevent growth one needs
  - To **decrease** the selection probability for **bigger than average programs**
  - To **increase** the selection probability for **below-average-size programs**
  - Or both

Bloat Control with Fitness Holes

- If one can create artificial fitness holes in areas of the search space which we want to avoid, then evolution will try to stay away from those areas.



Parsimony Pressure Method

- A **term** is added to the **fitness** function that **penalises** larger programs. Typically:

$$f_{parsimony}(prog) = f_{raw}(prog) - \mathbf{c} * size(prog)$$

where **c** is a constant (**parsimony coefficient**)

- An alternative

$$f_{parsimony}(prog) = f_{raw}(prog) + c/size(prog)$$

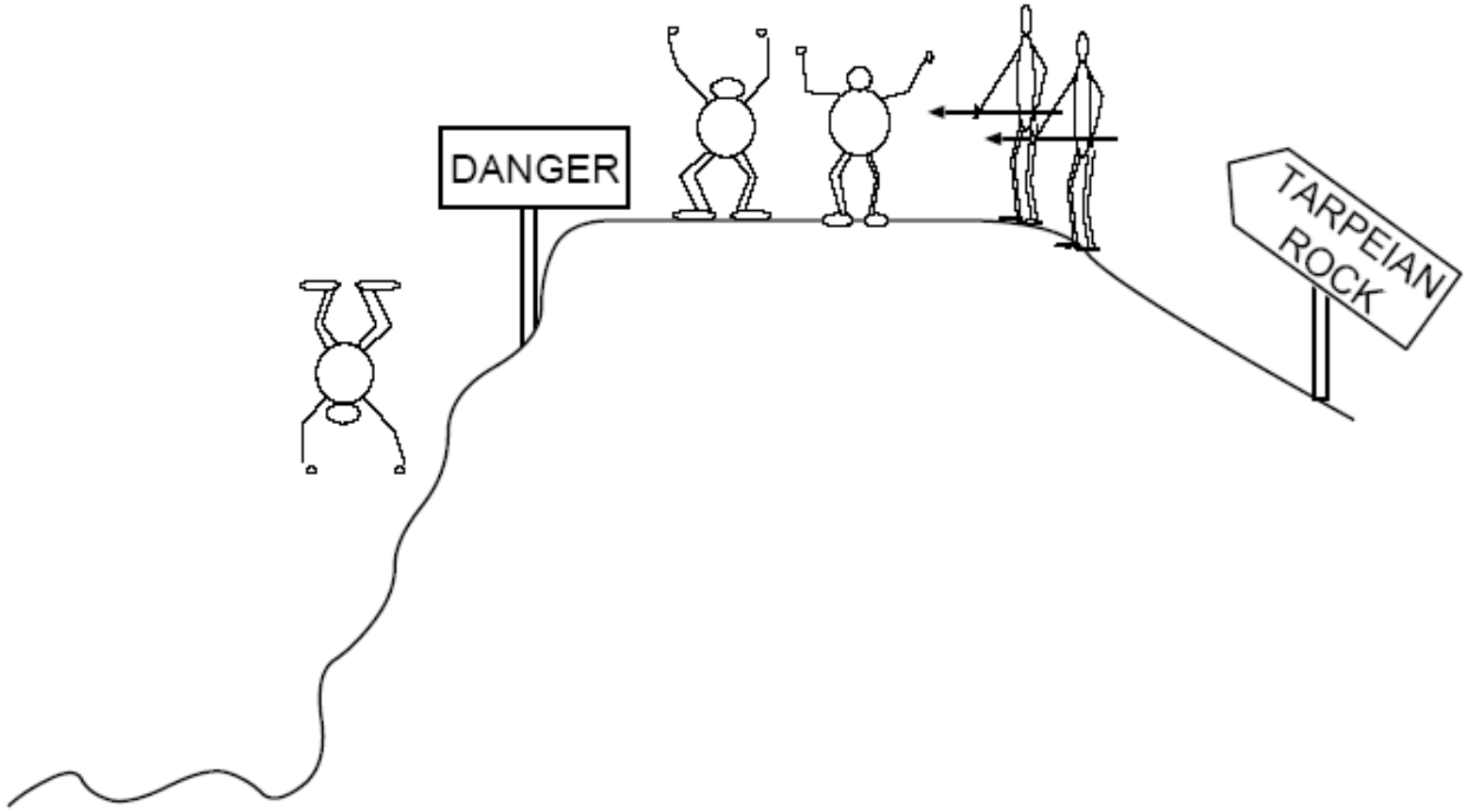
Tarpeian bloat control

- The basic Tarpeian method decreases the selection probability of longer-than-average programs, but it does it for **some random individuals** in the set and **for some of the time**
- Tarpeian fitness-wrapper

```
IF (size(program) > average_pop_size) AND (rand_num < ptarp) THEN
  return( very_low_fitness );
ELSE
  return( fitness(program) );
```

- The Tarpeian method creates **fitness holes dynamically and non-deterministically** (only for larger-than-average programs).

Why “Tarpeian”?



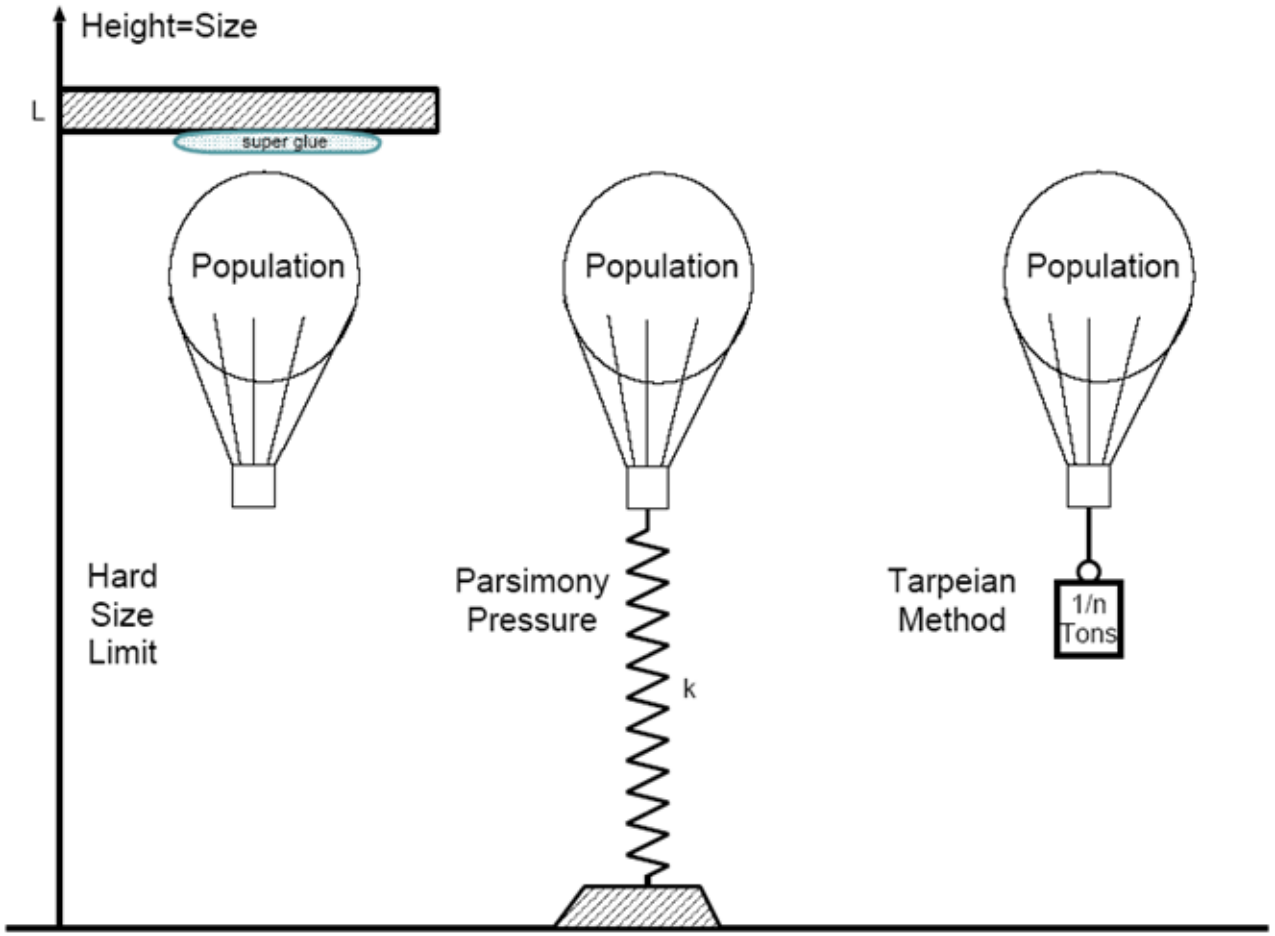
Features of the Tarpeian Method (1)

- The **wrapper does not evaluate** the individuals killed, so **computation** is **saved** not wasted.
- If enough better-than-average longer-than-average individuals are produced, the average program size grows. That is, the **algorithm allows growth as long as it is associated to fitness improvements**.

Features of the Tarpeian Method (2)

- If the average program size increases, the **fitness hole moves** so as to discourage further growth.
- The parameter p_{tarp} determines the intensity of the repulsive force exerted by the fitness hole.
- The **pressure to shrink does not change with program size**.

Hot Air Balloon Metaphor



Covariant parsimony pressure

- Parsimony pressure method

$$f_p = f - c \cdot \mathrm{s}$$

- Size evolution equation can be rewritten as:

$$E[\Delta\mu] = \frac{Cov(f,s) - c \cdot Var(s)}{\bar{f} - c \cdot \mu}$$

- If we want no growth, then we can set the right-hand side to 0. Solving for c gives

$$\mathrm{c} = \frac{Cov(f,s)}{Var(s)}$$

What did we learn? (1)

- We have learned that **bloat** is the **growth** in **program size** (variable length representation) without **significant** return in terms of **fitness**.
- **Replication accuracy theory:** GPs evolve bloated representations that are functionally like the parents.
- **Removal bias theory:** crossover inserts subtrees that on average are bigger than excised ones
- **Crossover bias theory:** Crossover creates many small and unfit programs.

What did we learn? (2)

- Growth can only occur when the selection probability of **short** programs is **lower** than average and of **long** programs is **higher** than average.
- Limit bloat by using a fixed size or depth limit or modifying the genetic operators
- Prevent bloat by adding a penalty term to the fitness (**parsimony pressure**) or by dynamically and non-deterministically creating fitness holes (**Tarpeian**)

[CE310] Evolutionary Computation and Genetic Programming

Riccardo Poli

••••

University of Essex
School of Computer Science and Electronic Engineering